



Université Hassan II
Ecole Supérieure de Technologie de Casablanca
Filière : Génie Informatique

RAPPORT DE STAGE D'INITIATION

PRÉPARÉ PAR : DAHI Babe Saleh
SOCIÉTÉ : DIRECTION GÉNÉRALE DU
TRÉSOR ET DE LA COMPTABILITÉ
PUBLIQUE
ANNÉE UNIVERSITAIRE : 2021 -2022







REMERCIEMENTS

C'est un devoir agréable d'exprimer en quelques lignes la reconnaissance qu'on doit à tous ceux dont on a sollicité l'aide et la Collaboration.

Nous souhaitons remercier, le corps professoral, de nous avoir bien accueilli au sein de cet établissement et la formation de qualité qu'ils nous ont prodiguée durant cette année, et spécialement M. Mohamed OUZZIF et M. Khalid BOURAGBA pour tous les efforts extrascolaires et administratifs qu'ils ont veillé à prodiguer sans retenue.

Nous remercions Dieu tout-Puissant de nous avoir permis de mener à terme ce travail qui est pour nous le point de départ d'une merveilleuse aventure, celle de la recherche, source de remise en cause permanente et de perfectionnement perpétuel.

Merci.



Table de matières

REMERCIEMENTS.....	3
Table matières.....	4
Rubriques du rapport	5
CHAPITRE I : PRÉSENTATION DE LA DIRECTION	6
1- Direction Générale de Trésor et de la Comptabilité publique.....	7
2- L'Organisation.....	8
3- Les contacts	10
CHAPITRE II : APPLICATION WEB POUR LA GESTION DE STOCK AU SEIN DE LA DGTCP.....	11
1 - Les buts de l'application.....	12
2 - Modèle conceptuel des données.....	13
CHAPITRE III : LA RÉALISATION.....	14
1 - l'architecture de l'application.....	15
3 - Les obstacles rencontrés	21
CHAPITRE IV : LES LOGICIELS ET FRAMEWORKS	23
1 - Flutter.....	24
2 - Dart et Python.....	25
2 - Django	26
3 - REST APIs et JSON	28
4 - HTTP.....	29
CHAPITRE V : CONCLUSION.....	30
1 - Ce que j'ai appris	31



Les rubriques du rapport

0 1	CHAPITRE II	une brève introduction à la DGTCP
0 2	CHAPITRE II	une brève présentation du projet
0 3	CHAPITRE III	outils et langages utilisés dans la réalisation du projet
0 4	CHAPITRE IV	comment le projet a été réalisé
0 5	CONCLUSION	Une conclusion



CHAPITRE I : PRÉSENTATION DE LA DIRECTION





INTRODUCTION

Du 17 juillet 2021 au 16 septembre 2021, j'ai effectué un stage à la Direction Générale du Trésor et de la Comptabilité Publique (**DGTCP**) à Nouakchott. Lors de ce stage au sein du service informatique, j'ai pu se concentrer sur le développement d'applications Web.

Plus largement, ce stage a été une opportunité professionnelle pour bien appréhender comment fonctionne le monde du génie logiciel, cela me permet aussi de découvrir le monde de l'entreprise et la mise en pratique des connaissances acquises à l'école Supérieur de la technologie.

Au-delà d'enrichir mes connaissances en informatique, ce stage m'a permis de comprendre dans quelle mesure je peux m'intégrer au travail d'équipe et découvrir mes vrais talents.

Direction Générale du Trésor et de la Comptabilité Publique

La direction générale du trésor et de la comptabilité publique est créée en vertu de la loi 62-133 du 29 juin 1962 portant la constitution et la réglementation du trésor.

Elle est chargée d'assurer l'exécution des dépenses de l'Etat, au niveau central et déconcentré, et de comptabiliser les dépenses et recettes exécutées, en vue de justifier l'emploi des deniers publics auprès de la représentation nationale (contrôle parlementaire) sous la surveillance de la Cour des Comptes (contrôle juridictionnel).

A ce titre, la Direction Générale du Trésor et de la Comptabilité Publique doit veiller au respect des textes relatifs à la réglementation et au contrôle de la dépense publique. Ses principales missions se portent sur :

- L'exécution en recettes et en Dépenses du Budget de l'Etat et de la Centralisation des Comptes ;
- L'élaboration des règles de la Comptabilité Publique ;
- La recherche et de la gestion des moyens de trésorerie ;
- La tenue des comptes de la Caisse des Dépôts et Consignations ;
- La tenue de la Caisse des Retraites, des collectivités locales, des établissements publics et des particuliers ;
- La gestion du portefeuille de l'Etat.



L'organisation de la DGTCP

L'administration centrale de la DGTCP comprend :

- La Direction de la Centralisation Comptable ;
- La Direction de la Gestion de la Trésorerie ;
- La Direction des Etudes et du Système d'Information ;
- La Direction des Finances Locales ;
- la Direction de l'Audit et du Contrôle Interne ;
- la Direction de Ressources Humaines et Moyens Généraux ;
- La Direction Régionale de Nouadhibou ;
- Les Trois Paeries ;
- Les Départements Comptables Ministériels.

Les structures rattachées à la Direction Générale sont :

- le service de l'information, renseignements et bureau d'ordre ;
- les perceptions de Nouakchott dépendant de la DGTCP.

La DCC compte 4 services : le service de la centralisation, le service de la comptabilité, le service du recouvrement et le service de la reddition des comptes de l'Etat.

La DGT comprend 4 services : le service de la gestion de la trésorerie, le service de la gestion de la dette, le service dépôts de fonds Trésor et le service de la caisse centrale.

La DESI comprend 3 services : le service de la réforme comptable, le service du système d'information et le service des études et des statistiques.

La DFL comprend 2 services : le service des statistiques communales et de la formation des Receveurs municipaux et le service de la mise en état d'examen des comptes de gestion des Receveurs municipaux et de la qualité comptable.

La DRHMG comprend 2 services : le service des ressources humaines et le service des moyens généraux.



ACI regrouper 15 auditeurs qui ont rang de chef service et 15 inspecteurs vérificateurs qui ont rang de chef division.

La PGT comprend 5 services : le service des dépenses de matériel, le service des dépenses sur marchés et des mises à disposition, le service des dépenses de personnel, le service du compte de gestion et le service de la comptabilité.

La PDDE comprend 2 services : le service des agences et le service de la comptabilité.

La PPCDC comprend 2 services : le service des agences et le service de la comptabilité.
Les DCM verront leur organisation fixée ultérieurement.

Le réseau comptable comprend 117 postes comptables qui se répartissent ainsi : 15 trésoreries régionales, 60 perceptions et 42 postes comptables diplomatiques.





Les Contacts

- Email : tresor.rim2015@gmail.com
- Adresse : Ave Abdel Nasser, à proximité de Ministère De La Justice, Nouakchott, Mauritanie.
- Télé : 46 71 21 40 / 38 38 53 89
- Site Web : www.tresor.mr
- Facebook, LinkedIn: DGTCP





CHAPITRE II: APPLICATION WEB POUR LA GESTION DE STOCK





Les Buts

L'objectif global de l'application est de permettre à la direction de gérer efficacement ses propriétés physiques, des ordinateurs de bureau aux chaises, etc. Et d'ajouter une couche de traçabilité au stock.

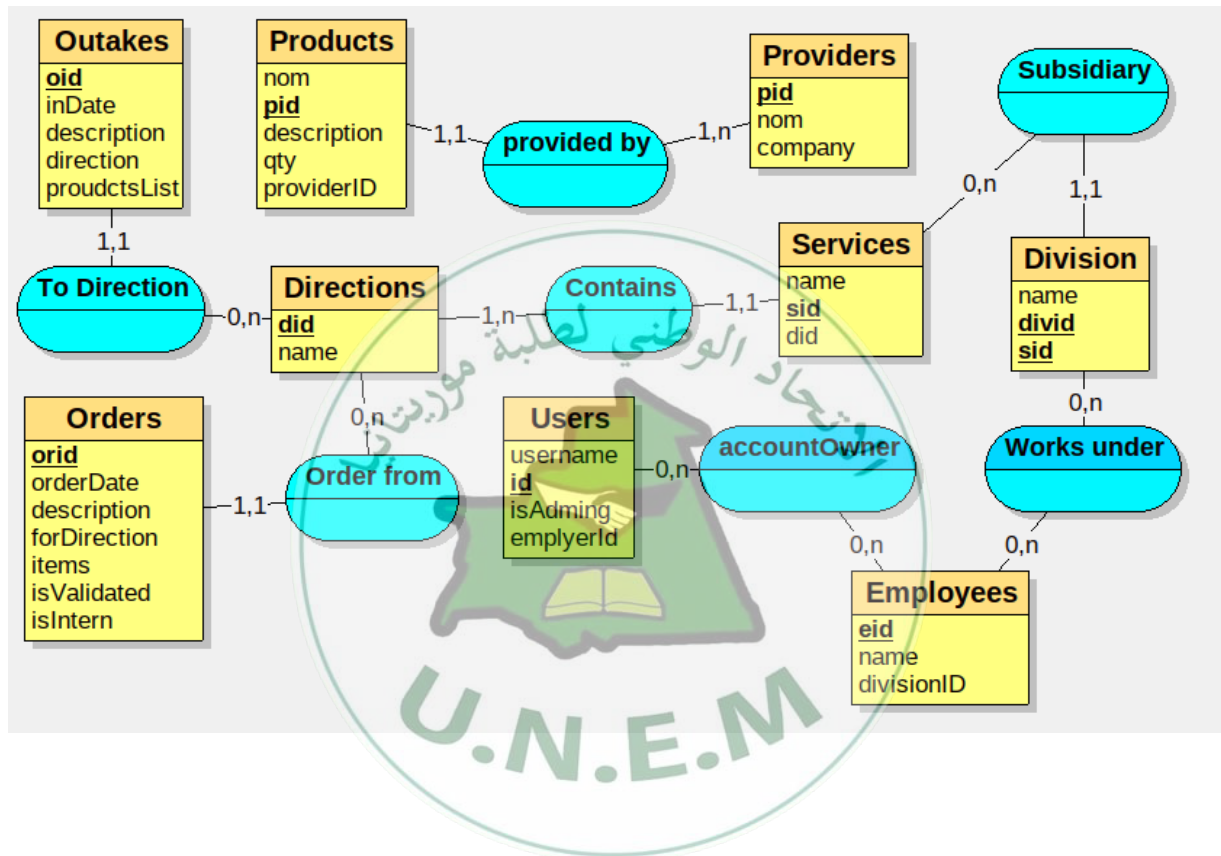
Mon mentor El Hassen Gvave m'a confié le projet et m'a donné la liberté de choisir la pile technique qui convient le mieux aux compétences que je veux acquérir et au projet.

Les exigences étaient les suivantes :

- L'application doit avoir une base de données contenant tout le personnel de chaque division, service et direction de l'administration. Toutes les propriétés physiques et les bureaux, tous logiquement connectés.
- Chaque division peut exprimer un besoin de (demander un produit).
- L'administrateur de stock peut valider ou refuser une "expression de besoin".
- Devrait avoir une liste de tous les produits dans l'inventaire.

À première vue, le projet semblait très complexe et énorme. Mais avec la modélisation de Merise et d'autres recherches, j'ai commencé à avoir une image complète du système.

Modèle conceptuel des données





CHAPITRE III : RÉALISATION



Architecture de l'application

J'ai conçu la couche de présentation (Frontend ou Client-side) avec Flutter. toutes les pages et les composants de l'interface utilisateur sont construits à l'aide des divers widgets de Flutter.

En utilisant Django, j'ai construit un serveur Web simple, intégré à la base de données PostgreSQL, qui sert les données à la couche de présentation.

La communication entre les deux (client et serveur) se fait avec une API REST : en utilisant des requêtes et des réponses HTTP, les données des réponses HTTP sont au format JSON, donc, à la fin, nous avons deux entités parlant le langage json en utilisant le fameux protocole internet HTTP.

En bref, l'application Flutter (le site Web) envoie simplement une HTTP GET requête au point de terminaison (Endpoint) de l'API. par exemple si le site Web a besoin d'une liste de produits de la base de données ? tout ce dont il a besoin est d'invoquer une fonction qui envoie une requête HTTP GET au point de terminaison, et en retour, lorsque ce point de terminaison est déclenché, l'application django (le serveur) enverra une réponse HTTP avec le corps contenant les données au format JSON, comme ceci :

```
114
115 Future<Product> getProductWithPid(int pid) async {
116     Uri uri = Uri.parse('${_url}products/$pid');
117     http.Response response;
118
119     response = await http.get(uri);
120     if (response.statusCode == 200) {
121         var data = jsonDecode(response.body);
122     }
```

plus simplement :



`http://127.0.0.1:8000/data/products/0`



autres exemples de l'API endpoints sont :

<http://127.0.0.1:8000/data/providers/0>

<http://127.0.0.1:8000/data/directions/0>

<http://127.0.0.1:8000/data/services/0>

les résultats seraient:

```
[
  {
    "model": "database.product",
    "pk": 1,
    "fields": {
      "name": "Herman Miller Sayl",
      "description": "Affordable ergonomics with a look you'll either love or hate",
      "providerID": 1,
      "qty": 12,
      "type": "d"
    }
  },
  {
    "model": "database.product",
    "pk": 2,
    "fields": {
      "name": "HP Envy 34 All-in-One",
      "description": "A great, and rare, big-screen all-in-one. $2,199 at Amazon",
      "providerID": 2,
      "qty": 20,
      "type": "c"
    }
  }
]
```

le zéro à la fin indique que nous voulons tous les éléments de la table de la base de données, sinon vous devez spécifier l'identifiant (**la clé primaire**) de l'entrée souhaitée à la place du zéro. par exemple : <http://127.0.0.1:8000/data/products/1>

Qui donnent uniquement l'entrée de table avec cet identifiant :



```
127.0.0.1:8000/data/products/1

Notion Google Calendar ActivityWatch

[
  {
    "model": "database.product",
    "pk": 1,
    "fields": {
      "name": "Herman Miller Sayl",
      "description": "Affordable ergonomics with a look you'll either love or hate",
      "providerID": 1,
      "qty": 12,
      "type": "d"
    }
  }
]
```

puis flutter analyse les données (désérialisation JSON) et les convertit en classes Dart pour une manipulation facile.

Sécurité:

à ce stade, vous pouvez déjà voir que cette architecture n'est pas sécurisée, toute personne disposant du point de terminaison de l'API peut obtenir des données du serveur ou même saboter la base de données en ajoutant des données inutiles à l'aide de la méthode GET POST.

Cela ajoute donc la nécessité d'un mécanisme d'autorisation : nous voulons que seuls les clients qui possèdent l'apiToken puissent interagir avec l'API.

j'ai donc implémenté une autorisation basée sur des jetons:

- dans l'en-tête de chaque requête HTTP, nous ajoutons une variable de jeton (apiToken).
- Ainsi, dans chaque méthode de réponse de notre application Server, il y a une vérification si la demande contient un en-tête apiToken et que cet apiToken est le même que l'apiToken pré-convenu. si tel est le cas, le serveur renvoie une réponse HTTP avec le code d'état 200 et les données, sinon il renvoie un code d'état de 401 et une chaîne 'Non autorisé'.

ce mécanisme qui est assez simple, ajoute une couche de sécurité nécessaire à l'application sans trop de complexité.

Démonstration:



code côté client:

```
9 static const String apiToken = '31048116';
10 static const String _url = 'http://127.0.0.1:8000/data/';
11 static const Map<String, String> headers = {
12   'apiToken': apiToken,
13 };
14
15 // Providers
16 Future<List<Providers>> getAllProviders() async {
17   List<Providers> list = [];
18   Uri uri = Uri.parse('${_url}providers/0');
19   http.Response response;
20   response = await http.get(uri, headers: headers);
21   if (response.statusCode == 200) {
22     . . .
23   }
```

- code côté serveur:

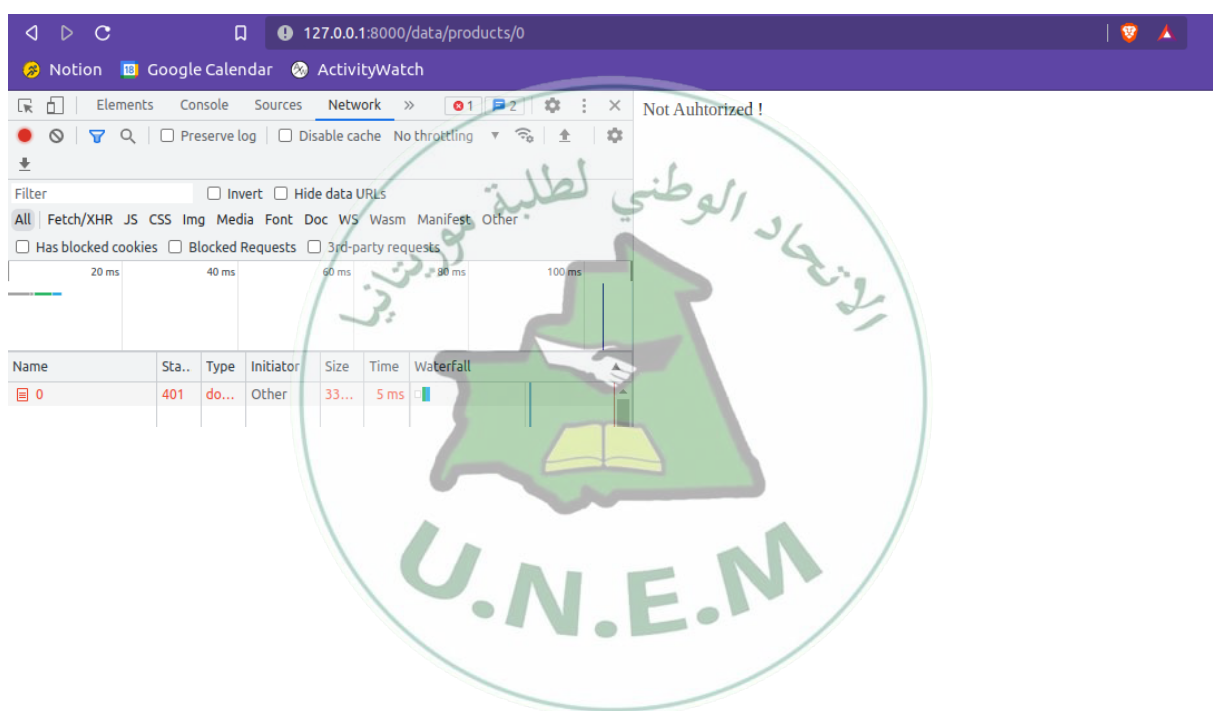
```
22 def get(self, request, pid):
23
24   apiToken = str(request.headers.get('apiToken'))
25   if apiToken == '31048116':
26     if pid == 0:
27       raw = Product.objects.all()
28       products = serializers.serialize('json', raw)
29       return HttpResponse(products, status=200, content_type="text/j
30   data = Product.objects.get(pid=pid)
31   product = serializers.serialize('json', [data])
32   return HttpResponse(product, status=200, content_type="text/json-c
33   return HttpResponse('Not Auhtorized !', status=401)
34
```

et après cela l'accès à l'API depuis le navigateur n'est plus possible :



Not Authorized !

Si nous enquêtons sur la requête à l'aide de la fonctionnalité « inspecter l'élément » du navigateur, nous constatons que la requête a échoué avec le code status 401 :





127.0.0.1:8000/data/products/0

Notion Google Calendar ActivityWatch

Elements Console Sources Network >> 1 2

Preserve log Disable cache No throttling

Filter Invert Hide data URLs

All Fetch/XHR JS CSS Img Media Font Doc WS Wasm Manifest Other

Has blocked cookies Blocked Requests 3rd-party requests

20 ms 40 ms 60 ms 80 ms 100 ms

Name Headers Preview Response Initiator >>

0

General

Request URL: http://127.0.0.1:8000/data/products/0

Request Method: GET

Status Code: 401 Unauthorized

Remote Address: 127.0.0.1:8000

Referrer Policy: strict-origin-when-cross-origin

الاتحاد الوطني لطلبة موريتانيا

U.N.E.M.



Les obstacles rencontrés

Dans la phase de programmation, j'ai rencontré de nombreux obstacles, les suivants ont été les plus difficiles :

- Liaison de Postgresql avec le serveur django :

dans le fichier django settings.py est l'endroit où vous spécifiez la base de données et sa configuration, j'ai rencontré un énorme problème de connexion à Postgres sur le port 5432, et après de nombreuses recherches, j'ai résolu le problème en modifiant les fichiers postgresql.conf et pg_hba.conf.

- En-tête de réponse HTTP Access-Control-Allow-Origin : L'en-tête

Access-Control-Allow-Origin est inclus dans la réponse d'un site Web à une requête provenant d'un autre site Web et identifie l'origine autorisée de la requête.

j'ai résolu cette erreur d'autorisation en ajoutant à toutes les requêtes HTTP :

```
headers: {
```



```
'Access-Control-Allow-Origin': '*',  
'Content-Type': 'application/json',  
  'Accept': '*/*'  
}
```





CHAPITRE IV : LES LOGICIELS ET FRAMEWORKS





Flutter



Flutter est un SDK (kit de développement logiciel) d'application mobile permettant de créer des applications hautes performances et haute fidélité pour iOS et Android.

Avec de puissantes bibliothèques de graphiques et d'animations, le framework Flutter facilite la création d'interfaces utilisateur qui réagissent en douceur au toucher.

Flutter est basé sur le langage de programmation Dart et fournit un flux de travail de développement rapide avec rechargement à chaud, ce qui vous permet d'itérer rapidement sur votre code.

Certaines des principales fonctionnalités de Flutter incluent :

- Langage de programmation **Dart** : Flutter utilise le langage de programmation Dart, qui est facile à apprendre et vous permet de développer des applications de haute qualité.
- Rechargement à chaud : la fonction « **rechargement à chaud** » de Flutter vous permet d'apporter rapidement et facilement des modifications à votre application sans la redémarrer.
- Interface utilisateur expressive et flexible : les éléments de l'interface utilisateur de Flutter sont construits selon les mêmes principes que les directives de conception matérielle de Google, vous offrant un moyen expressif et flexible de créer de belles applications.
- Performances natives : les applications Flutter sont compilées en code natif, vous offrant les meilleures performances possibles sur iOS et Android.
- Open source : Flutter est un projet open source, ce qui signifie que vous pouvez l'utiliser gratuitement et contribuer au développement de la plateforme.

Pourquoi j'ai choisi Flutter :

- J'ai déjà de l'expérience dans la création d'applications dans Flutter.
- le développement frontend avec Flutter est rapide, j'aurai donc le temps de me concentrer sur le backend.
- Je peux créer une application de bureau et un site Web à partir de la même base de code.



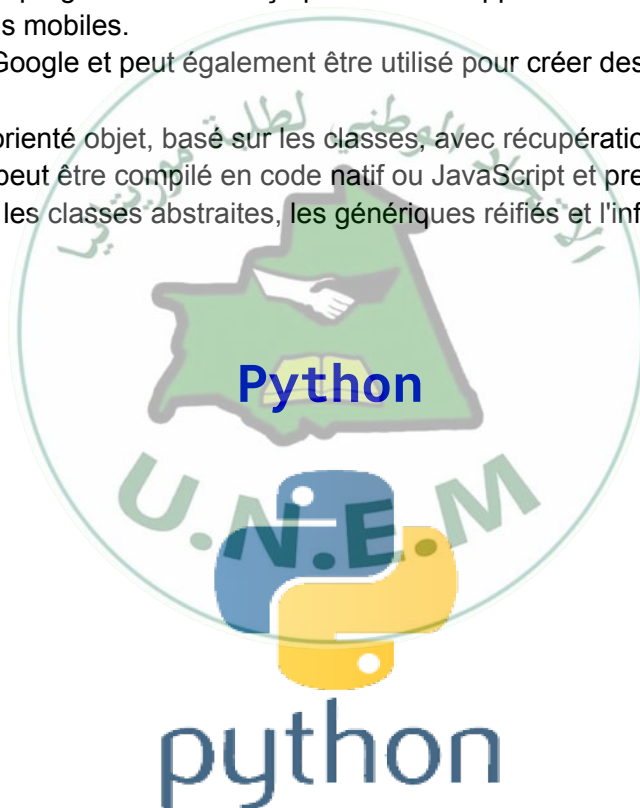
Dart



Dart est un langage de programmation conçu pour le développement client, comme pour le Web et les applications mobiles.

Il est développé par Google et peut également être utilisé pour créer des applications serveur et de bureau.

Il s'agit d'un langage orienté objet, basé sur les classes, avec récupération de place avec une syntaxe de style C. Il peut être compilé en code natif ou JavaScript et prend en charge les interfaces, les mixins, les classes abstraites, les génériques réifiés et l'inférence de type.



Python est un langage de programmation généraliste de haut niveau.

Sa philosophie de conception met l'accent sur la lisibilité du code avec l'utilisation d'une indentation significative.

Python est typé dynamiquement et ramassé. Il prend en charge plusieurs paradigmes de programmation, y compris la programmation structurée, orientée objet et fonctionnelle.



Django



Django est un framework Web Python de haut niveau qui permet le développement rapide de sites Web sécurisés et maintenables. Construit par des développeurs expérimentés, Django prend en charge une grande partie des tracas du développement Web, vous pouvez donc vous concentrer sur l'écriture de votre application sans avoir à réinventer la roue. Il est gratuit et open source, possède une communauté florissante et active, une excellente documentation et de nombreuses options de support gratuit et payant.

Django vous aide à écrire des logiciels qui sont :

Complet

Django suit la philosophie "Batteries incluses" et fournit presque tout ce que les développeurs pourraient vouloir faire "prêt à l'emploi". Parce que tout ce dont vous avez besoin fait partie d'un seul "produit", tout fonctionne parfaitement ensemble, suit des principes de conception cohérents et dispose d'une documentation complète et à jour.

Polyvalent

Django peut être (et a été) utilisé pour construire presque n'importe quel type de site Web - des systèmes de gestion de contenu et des wikis, aux réseaux sociaux et aux sites d'actualités. Il peut fonctionner avec n'importe quel framework côté client et peut fournir du contenu dans presque tous les formats (y compris HTML, flux RSS, JSON et XML).

En interne, bien qu'il offre des choix pour presque toutes les fonctionnalités que vous pourriez souhaiter (par exemple, plusieurs bases de données populaires, moteurs de modèles, etc.), il peut également être étendu pour utiliser d'autres composants si nécessaire.

Sécurisé

Django aide les développeurs à éviter de nombreuses erreurs de sécurité courantes en fournissant un cadre qui a été conçu pour « faire ce qu'il faut » pour protéger automatiquement le site Web. Par exemple, Django fournit un moyen sécurisé de gérer les comptes d'utilisateurs et les mots de passe, en évitant les erreurs courantes comme mettre les informations de session dans les cookies là où elles sont vulnérables (au lieu de cela, les cookies ne contiennent qu'une clé et les données réelles sont stockées dans la base de données) ou stocker directement les mots de passe. plutôt qu'un hachage de mot de passe.



Django permet une protection contre de nombreuses vulnérabilités par défaut, notamment l'injection SQL, les scripts intersites, la falsification de requêtes intersites et le détournement de clics (voir Sécurité des sites Web pour plus de détails sur ces attaques).

Évolutif

Django utilise une architecture "shared-nothing" basée sur des composants (chaque partie de l'architecture est indépendante des autres, et peut donc être remplacée ou modifiée si nécessaire). Avoir une séparation claire entre les différentes parties signifie qu'il peut s'adapter à un trafic accru en ajoutant du matériel à n'importe quel niveau : serveurs de mise en cache, serveurs de base de données ou serveurs d'applications. Certains des sites les plus fréquentés ont réussi à faire évoluer Django pour répondre à leurs demandes (par exemple, Instagram et Disqus, pour n'en nommer que deux).

Maintenable

Le code Django est écrit en utilisant des principes de conception et des modèles qui encouragent la création de code maintenable et réutilisable. En particulier, il utilise le principe Don't Repeat Yourself (DRY) afin qu'il n'y ait pas de duplication inutile, ce qui réduit la quantité de code. Django promeut également le regroupement des fonctionnalités associées dans des "applications" réutilisables et, à un niveau inférieur, regroupe le code associé dans des modules (à l'instar du modèle Model View Controller (MVC)).

Portable

Django est écrit en Python, qui fonctionne sur de nombreuses plates-formes. Cela signifie que vous n'êtes lié à aucune plate-forme de serveur particulière et que vous pouvez exécuter vos applications sur de nombreuses versions de Linux, Windows et macOS. De plus, Django est bien pris en charge par de nombreux fournisseurs d'hébergement Web, qui fournissent souvent une infrastructure et une documentation spécifiques pour l'hébergement de sites Django.



REST APIs et JSON

Qu'est-ce qu'une API ?

Un API est un ensemble de définitions et de protocoles permettant de créer et d'intégrer des logiciels d'application. On parle parfois de contrat entre un fournisseur d'informations et un utilisateur d'informations, établissant le contenu requis du consommateur (l'appel) et le contenu requis par le producteur (la réponse). Par exemple, la conception de l'API pour un service météo pourrait spécifier que l'utilisateur fournisse un code postal et que le producteur réponde par une réponse en 2 parties, la première étant la température élevée et la seconde la basse.

En d'autres termes, si vous souhaitez interagir avec un ordinateur ou un système pour récupérer des informations ou exécuter une fonction, une API vous aide à communiquer ce que vous voulez à ce système afin qu'il puisse comprendre et répondre à la demande.

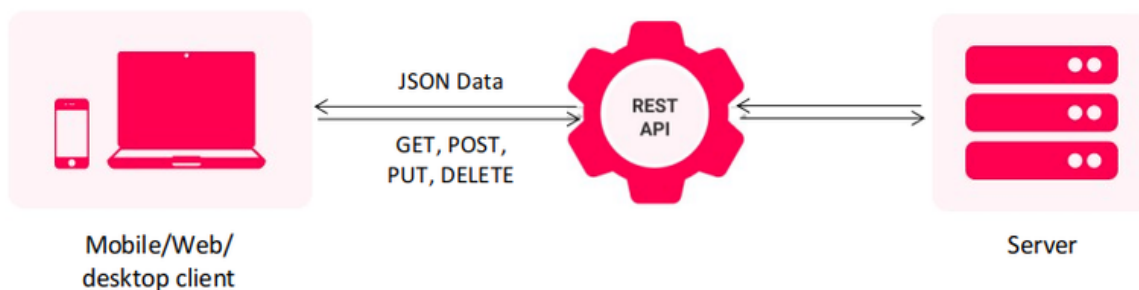
Vous pouvez considérer une API comme un médiateur entre les utilisateurs ou les clients et les ressources ou les services Web qu'ils souhaitent obtenir. C'est également un moyen pour une organisation de partager des ressources et des informations tout en maintenant la sécurité, le contrôle et l'authentification, en déterminant qui a accès à quoi.

Un autre avantage d'une API est que vous n'avez pas besoin de connaître les spécificités de la mise en cache, c'est-à-dire comment votre ressource est récupérée ou d'où elle vient.

Qu'est-ce qu'une API REST ?

Une API REST (également connue sous le nom d'API RESTful) est une interface de programmation d'application (API ou API Web) qui se conforme aux contraintes du style architectural REST et permet une interaction avec les services Web RESTful. REST signifie transfert d'état représentatif et a été créé par l'informaticien Roy Fielding.

REST API Model





HTTP

Le protocole de transfert hypertexte (HTTP) est un protocole de couche application dans le modèle de suite de protocoles Internet pour les systèmes d'information distribués, collaboratifs et hypermédia. HTTP est la base de la communication de données pour le World Wide Web, où les documents hypertextes incluent des hyperliens vers d'autres ressources auxquelles l'utilisateur peut facilement accéder, par exemple par un clic de souris ou en touchant l'écran dans un navigateur Web.

Le développement de HTTP a été initié par Tim Berners-Lee au CERN en 1989 et résumé dans un simple document décrivant le comportement d'un client et d'un serveur utilisant la première version du protocole HTTP nommée 0.9.

Cette première version du protocole HTTP a rapidement évolué vers une version plus élaborée qui était la première ébauche vers une future version 1.0.





CHAPITRE V : CONCLUSION





Ce que j'ai appris

durant le mois de stage à la DGTCP, j'ai rencontré beaucoup de professionnels auprès desquels j'ai beaucoup appris.

J'ai appris des trucs techniques et aussi des trucs liés au travail avec les gens et à l'intégration dans l'environnement de l'entreprise.

voici les :

conseils techniques :

- toujours diviser votre travail ou projet en plus petits morceaux pour une meilleure gestion de projet.
- toujours mettre des délais pour chaque tâche. ou bien il pourrait s'étendre à n'importe combien de temps il y a.
- structurez toujours votre code et recherchez la simplicité.

conseils sociaux :

- respecter les gens et être à votre parole.
- assumer la responsabilité de vos actes.
- montrez un réel intérêt pour ce que vous faites et recherchez des opportunités d'apprentissage.

